

189. 轮转数组

1.question

给定一个整数数组 `nums`，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。

示例 1:

```
输入: nums = [1,2,3,4,5,6,7], k = 3
输出:[5,6,7,1,2,3,4]解释:
向右轮转 1 步:[7,1,2,3,4,5,6]
向右轮转 2 步:[6,7,1,2,3,4,5]
向右轮转 3 步:[5,6,7,1,2,3,4]
```

示例 2:

```
输入: nums = [-1,-100,3,99], k = 2
输出: [3,99,-1,-100]
解释:
向右轮转 1 步: [99,-1,-100,3]
向右轮转 2 步: [3,99,-1,-100]
```

提示:

- `1 <= nums.length <= 105`
- `-231 <= nums[i] <= 231 - 1`
- `0 <= k <= 105`

进阶:

- 尽可能想出更多的解决方案，至少有 **三种** 不同的方法可以解决这个问题。
- 你可以使用空间复杂度为 `O(1)` 的 **原地** 算法解决这个问题吗？

2.解决方法

好的，我们来分析“轮转数组”这个问题 (LeetCode 189)。

1. 问题理解

- **目标：** 将一个数组中的元素向右轮转（移动）指定的步数 `k`。
- **输入：** 一个整数数组 `nums` 和一个非负整数 `k`（轮转的步数）。
- **输出：** 没有显式返回值，需要原地修改输入的数组 `nums`。
- **原地修改：** 意味着我们应该尽量不使用额外的、与数组大小成比例的存储空间来完成操作（目标是 $O(1)$ 额外空间）。
- **`k` 的值：** `k` 可能大于数组的长度。旋转 `n` 步等于没旋转，所以实际有效的旋转步数是 `k % n`（`n` 是数组长度）。

2. 普通人的思路（模拟/额外数组）

- **思路一：一步一步转（模拟）**
 - 最直观的想法就是模拟旋转的过程。旋转 `k` 步，就是重复 `k` 次“旋转 1 步”的操作。
 - 旋转 1 步的操作：
 1. 记住数组最后一个元素 `last = nums[n-1]`。
 2. 将前面所有元素向右移动一位：`nums[i] = nums[i-1]`（从后往前移，`i` 从 `n-1` 到 `1`）。
 3. 将记住的 `last` 放到数组开头：`nums[0] = last`。
 - 重复这个“旋转 1 步”的操作 `k` 次。
 - **缺点：** 如果 `k` 很大（比如接近 `n`），总的操作次数是 `k * n`，时间复杂度是 $O(n*k)$ ，当 `n` 和 `k` 都很大时会超时。空间复杂度 $O(1)$ 。
- **思路二：使用额外数组**
 - 创建一个和 `nums` 一样大小的新数组 `newNums`。
 - 遍历原数组 `nums`，对于每个元素 `nums[i]`，计算它旋转 `k` 步后的新位置。新位置应该是 `(i + k) % n`（`n` 是数组长度，取模 `%` 处理循环）。
 - 将 `nums[i]` 放到新数组的对应位置：`newNums[(i + k) % n] = nums[i]`。
 - 遍历完后，`newNums` 里就是旋转后的结果。

- 最后，将 `newNums` 的内容复制回原数组 `nums`。
- **优点：** 时间复杂度是 $O(n)$ ，因为只需要遍历数组两次（一次计算放入新数组，一次复制回来）。
- **缺点：** 需要一个额外的数组，空间复杂度是 $O(n)$ ，不满足“原地修改”且 $O(1)$ 额外空间的要求。

3. 更好的方法（数组反转 - $O(1)$ 空间）

这是一种非常巧妙且高效的原地算法：

1. **处理 `k`：** 由于旋转 `n` 步等于没旋转，实际有效的旋转步数是 `k % n`。先计算 `k = k % nums.length`。如果 `k` 变成 0，则无需旋转，直接返回。
2. **三次反转：**
 - **第一步：** 反转整个数组（从索引 0 到 `n-1`）。
 - **第二步：** 反转数组的前 `k` 个元素（从索引 0 到 `k-1`）。
 - **第三步：** 反转数组剩下的 `n-k` 个元素（从索引 `k` 到 `n-1`）。

为什么这样可行？

想象数组是

`[1, 2, 3, 4, 5, 6, 7]`，`k=3`。目标是 `[5, 6, 7, 1, 2, 3, 4]`。

- 整体反转：`[7, 6, 5, 4, 3, 2, 1]`
- 反转前 `k=3` 个：`[5, 6, 7, 4, 3, 2, 1]`（看，目标的前缀 `[5, 6, 7]` 已经就位了！）
- 反转后 `n-k=4` 个：`[5, 6, 7, 1, 2, 3, 4]`（目标的后缀 `[1, 2, 3, 4]` 也通过反转就位了！）

复杂度：

- 时间复杂度： $O(n)$ 。因为我们对数组（或其部分）进行了三次完整的遍历（反转操作本质上是遍历）。
- 空间复杂度： $O(1)$ 。我们只需要几个临时变量来辅助反转，不需要额外的数组。

4. 示例演示（数组反转法）

`nums = [1, 2, 3, 4, 5, 6, 7]`，`k = 3`

1. `n = 7`。计算有效 `k`：`k = 3 % 7 = 3`。
2. 反转整个数组 `[0, 6]`：`[7, 6, 5, 4, 3, 2, 1]`
3. 反转前 `k=3` 个元素 `[0, 2]`：`[5, 6, 7, 4, 3, 2, 1]`
4. 反转后 `n-k=4` 个元素 `[3, 6]`：`[5, 6, 7, 1, 2, 3, 4]`

5. 完成！数组 `nums` 现在是 `[5, 6, 7, 1, 2, 3, 4]`。

5. 代码实现（数组反转法）

```
class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        // 1. 处理 k，使其在 [0, n-1] 范围内
        k %= n;
        if (k == 0) {
            return; // 无需旋转
        }

        // 2. 三次反转
        reverse(nums, 0, n - 1); // 反转整个数组
        reverse(nums, 0, k - 1); // 反转前 k 个元素
        reverse(nums, k, n - 1); // 反转后 n-k 个元素
    }

    /**
     * 辅助方法：反转数组 nums 的指定区间 [start, end]
     * @param nums 数组
     * @param start 起始索引（包含）
     * @param end 结束索引（包含）
     */
    private void reverse(int[] nums, int start, int end) {
        while (start < end) {
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start++;
            end--;
        }
    }
}

// @lc code=end
```

```

/*
// @lcpr case=start
// [1,2,3,4,5,6,7]\\n3\\n
// @lcpr case=end

// @lcpr case=start
// [-1,-100,3,99]\\n2\\n
// @lcpr case=end

*/

```

6. 如何在面试中讲解

- 确认问题：** "明白，我们需要将数组向右原地轮转 k 步。需要注意 k 可能大于数组长度，所以实际旋转步数是 $k \% n$ 。目标是 $O(1)$ 额外空间。 "
- 提几种思路：** "可以考虑几种方法：
 - 最直接的是模拟旋转 k 次，每次移动 1 步，但时间复杂度 $O(nk)$ 可能太高。
 - 可以用一个额外的 $O(n)$ 空间数组，计算每个元素的新位置，时间 $O(n)$ ，但不满足 $O(1)$ 空间要求。
 - 还有一种原地 $O(n)$ 时间、 $O(1)$ 空间的方法是使用**数组反转**。" (也可以提一下环状替换，但说明反转法通常更易实现)
- 详述反转法：** "反转法的思路很巧妙。它包含三个步骤：
 - 首先，将整个数组完全反转。
 - 然后，反转数组的前 k 个元素。
 - 最后，反转数组剩下的 $n-k$ 个元素。 "
- 解释原理/示例：** "我们可以通过一个例子来看，比如 $[1,2,3,4,5]$ 右旋 2 步。目标是 $[4,5,1,2,3]$ 。
 - 整体反转得到 $[5,4,3,2,1]$ 。
 - 反转前 $k=2$ 个得到 $[4,5,3,2,1]$ 。
 - 反转后 $n-k=3$ 个得到 $[4,5,1,2,3]$ 。正好是结果！ "
- 复杂度：** "这个方法只需要对数组进行三次线性扫描（反转），所以时间复杂度是 $O(n)$ 。反转是在原地进行的，只需要常数个额外变量，空间复杂度是 $O(1)$ 。 "

6. **实现细节：** "实现时需要一个辅助函数来反转数组的指定部分，通过双指针交换元素即可。"

这样讲解能清晰地展示你对不同方法的了解、对最优解的掌握以及其背后的逻辑。

找到具有 2 个许可证类型的类似代码